

Part A • 5-mark questions | Part B • 15-mark questions

Primary study notes • A4 print format

TABLE OF CONTENTS

PART A — 5-Mark Questions (1–16)

PART B — 15-Mark Questions (1–20)

FINAL REVISION CHECKLIST

PART A

5-MARK QUESTIONS

QUESTION 1

What are the three main layers in the architecture of an Artificial Neural Network (ANN), and what role do weights play in connecting these layers?

ANSWER

An Artificial Neural Network (ANN) is commonly organized into three main types of layers:

1. Input layer – receives the input features. It usually contains one neuron for each input feature.
2. Hidden layer(s) – perform intermediate computations. An ANN may have one or many hidden layers.

3. Output layer – produces the final prediction or classification result.

Weights are numerical parameters associated with the connections between neurons. Each input is multiplied by its

corresponding weight, and the weighted values are combined with a bias before an activation function is applied.

For a neuron:

$$z = \sum(w_i x_i) + b$$

$$y = f(z)$$

where x_i is an input, w_i is its weight, b is the bias, f is the activation function, and y is the neuron output.

Simple architecture:

Input Layer → Hidden Layer(s) → Output Layer

$x_i \rightarrow$

$x_i \rightarrow$ [Hidden neurons] → [Output]

$x_i \rightarrow$

During training, weights are adjusted using an optimization method so that the network minimizes its loss. Thus, weights

determine how strongly each input contributes to the computation performed by the next layer.

QUESTION 2

What are the formulas for Sigmoid and Softmax activation functions, and how do they differ in their applications?

ANSWER

The Sigmoid and Softmax functions are activation functions used to convert neural-network outputs into useful ranges or probabilities.

1. Sigmoid:

$$\sigma(z) = 1 / (1 + e^{-z})$$

Its output lies between 0 and 1. It is commonly used in the output layer for binary classification, where the output can be

interpreted as the estimated probability of the positive class.

2. Softmax:

$$\text{softmax}(z_i) = e^{z_i} / \sum_j e^{z_j}$$

For a vector of logits, Softmax converts all class scores into non-negative values whose sum is 1. It is commonly used for

multi-class classification when each example belongs to one of several mutually exclusive classes.

Difference:

- Sigmoid is typically used for binary classification or independently modeled multi-label outputs.
- Softmax is typically used for mutually exclusive multi-class classification.
- Sigmoid treats each output independently; Softmax produces a normalized probability distribution across the classes.

Example: for three mutually exclusive classes—cat, dog, bird—Softmax may produce [0.70, 0.20, 0.10], whose values sum

to 1.

QUESTION 3

What is feature engineering, and what are some common feature engineering techniques with examples?

ANSWER

Feature engineering is the process of transforming raw data into useful input features that help a machine-learning or

deep-learning model learn meaningful patterns.

Common techniques include:

1. Handling missing values – replace missing numerical values with an appropriate statistic such as the median, or use a model-based method.
2. Encoding categorical data – convert categories into numerical representations. Example: “Red, Blue, Green” can be one-hot encoded.

3. Scaling/normalization – transform numerical features to comparable ranges. Standardization is commonly:

$$z = (x - \mu) / \sigma$$

4. Feature extraction – derive informative features from raw data. Example: extract year, month, and day from a date.

5. Text features – convert text into numerical features using methods such as TF-IDF, word embeddings, or token counts.

6. Image features – resize images, normalize pixel values, or use learned CNN representations.

7. Feature construction – combine existing variables. Example: `total_amount = quantity × unit_price`.

Example: In a customer dataset containing age, city, income, and purchase date, age and income may be standardized,

city may be encoded, and purchase date may be transformed into month and weekday.

Good feature engineering can improve model learning, reduce irrelevant information, and make patterns easier for the model to identify.

QUESTION 4

What is convolution in a Convolutional Neural Network (CNN), how does the convolution operation work, and can you illustrate it with a suitable example?

ANSWER

Convolution is the operation through which a CNN applies a small learnable filter (kernel) to an input such as an image to detect local patterns such as edges, textures, and shapes.

A kernel slides across the input. At each position, corresponding values are multiplied and summed. The resulting value becomes one element of the feature map.

For a 2×2 kernel:

$$\text{output} = \sum \sum X \rightarrow K \rightarrow$$

Example:

Input patch:

[1 2

3 4]

Kernel:

[1 0
0 1]

Convolution result:

$$(1 \times 1) + (2 \times 0) + (3 \times 0) + (4 \times 1) = 5$$

The kernel then moves according to the chosen stride and repeats the calculation across the image, producing a feature map.

Typical CNN flow:

Input Image → Convolution/Filters → Activation → Pooling → Convolution → ... → Flatten → Dense → Output

The filters are learned during training. Different filters can learn different features, with early layers often learning simple local patterns and deeper layers combining them into more complex representations.

QUESTION 5

What are the major applications of Recurrent Neural Networks (RNNs)?

ANSWER

Recurrent Neural Networks (RNNs) are designed for sequential or time-dependent data because their hidden state carries information from previous time steps.

Major applications include:

1. Natural Language Processing (NLP) – language modeling, sentiment analysis, and sequence processing.
2. Speech recognition – processing sequences of audio features.
3. Machine translation – mapping a sequence in one language to a sequence in another.
4. Time-series forecasting – predicting values such as demand, temperature, or financial indicators from historical

sequences.

5. Text generation – generating text one token at a time.

6. Handwriting recognition – processing sequences of pen movements or image features.

7. Video/activity recognition – analyzing temporal sequences of visual features to recognize activities.

Basic recurrent idea:

$x_t \rightarrow [\text{RNN cell}] \rightarrow h_t \rightarrow \text{output}_t$

↑

h_{t-1}

The current hidden state depends on the current input and the previous hidden state. This makes RNNs suitable for data in

which order and temporal context matter.

QUESTION 6

What are the input, forget, and output gates in a Long Short-Term Memory (LSTM) network, and what is the function of each gate?

ANSWER

An LSTM is a recurrent architecture designed to retain useful information over long sequences. It uses gates to control

information flow.

1. Forget gate:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

It determines which information from the previous cell state should be discarded.

2. Input gate:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

It controls how much new information should enter the cell state.

Candidate cell state:

$$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

Cell-state update:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

3. Output gate:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

It controls which information from the updated cell state is exposed as the hidden state.

Hidden state:

$$h_t = o_t \odot \tanh(C_t)$$

Here σ is the sigmoid function, $\tanh$ is the hyperbolic tangent, and $\odot$ denotes element-wise multiplication.

Thus, the forget gate removes unnecessary old information, the input gate controls new information, and the output gate controls the information passed to the next layer/time step.

QUESTION 7

What are the major data requirements for Natural Language Processing (NLP), and can you provide suitable examples?

ANSWER

NLP systems require suitable textual data and supporting information so that language can be processed and a model can learn the target task.

Major requirements include:

1. Sufficient data volume – the dataset should contain enough examples for the task. Example: thousands of labeled reviews for sentiment classification.
2. Relevant text – data should represent the language and domain of the application. Example: medical text for a medical NLP application.
3. Correct labels when supervised learning is used – e.g., reviews labeled positive, negative, or neutral.
4. Good data quality – remove or handle duplicates, corrupted records, irrelevant text, and excessive noise.
5. Consistent representation – text may need encoding, tokenization, normalization, and handling of punctuation or special

characters.

6. Vocabulary/domain coverage – important words, terminology, abbreviations, and expressions should be represented.
7. Balanced or appropriately sampled classes – avoid severe class imbalance when it can distort training.
8. Train, validation, and test sets – separate data is required for training, model selection, and final evaluation.

Example: For spam-email classification, the dataset should contain email text plus spam/ham labels, cover representative

email language, remove problematic duplicates, preprocess the text consistently, and be divided into training, validation,

and test sets.

QUESTION 8

Which Python and NLP packages can be used to prepare text data for model development, and how are they applied with

suitable examples?

ANSWER

Several Python and NLP packages can prepare text for model development.

1. NLTK – provides tokenization, stop-word lists, stemming, lemmatization, and other NLP utilities.

Example:

```
from nltk.tokenize import word_tokenize  
tokens = word_tokenize("Deep learning is useful.")
```

2. spaCy – provides efficient tokenization, linguistic processing, lemmatization, and named-entity recognition.
3. pandas – used to load, inspect, clean, and manipulate tabular text datasets.

Example:

```
df = pd.read_csv("reviews.csv")
```

4. scikit-learn – provides TF-IDF vectorization and classical preprocessing tools.

Example:

```
from sklearn.feature_extraction.text import TfidfVectorizer  
vectorizer = TfidfVectorizer()  
X = vectorizer.fit_transform(df["review"])
```

5. TensorFlow/Keras – provides tokenization/vectorization and neural-network layers such as TextVectorization and

Embedding.

6. NumPy – supports numerical arrays and transformations used during preprocessing.

A typical workflow is:

Raw Text → Cleaning → Tokenization → Normalization → Feature Representation → Train/Validation/Test Data → Model

Example:

“Excellent product!” → tokens → normalized text → TF-IDF/embedding representation → classifier.

The appropriate package depends on the task, dataset size, and whether the final model is classical ML or deep learning.

QUESTION 9

What is the structure of a Perceptron model, and how can its working be illustrated with a suitable example?

ANSWER

A Perceptron is a basic single artificial neuron used for binary classification. It contains inputs, weights, a bias, a weighted-sum calculation, an activation function, and an output.

Structure:

$x_t \rightarrow w_i \rightarrow$

$x_t \rightarrow w_i \rightarrow \rightarrow \Sigma(w_i x_t) + b \rightarrow \text{Activation} \rightarrow y$

$x_t \rightarrow w_i \rightarrow$

Weighted sum:

$$z = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b$$

For a simple step activation:

$$y = 1, \text{ if } z \geq 0$$

$$y = 0, \text{ if } z < 0$$

Numerical example:

Let $x_t = 1$, $x_t = 0$, $w_i = 0.6$, $w_i = 0.4$, and $b = -0.5$.

$$z = (0.6)(1) + (0.4)(0) - 0.5$$

= 0.1

Since $z \geq 0$, the step activation gives $y = 1$.

During learning, the weights and bias are adjusted when the prediction is incorrect. A Perceptron can learn linearly

separable binary patterns, but a single Perceptron cannot represent non-linearly separable problems such as XOR.

QUESTION 10

What are loss functions in deep learning, and what are some common loss functions with suitable examples?

ANSWER

A loss function measures the difference between a model's prediction and the actual target. During training, the optimizer

attempts to minimize the loss by changing model parameters.

Common loss functions include:

1. Mean Squared Error (MSE) – commonly used for regression:

$$\text{MSE} = (1/n) \sum (y_i - \hat{y}_i)^2$$

2. Binary Cross-Entropy – commonly used for binary classification:

$$L = -[y \log(p) + (1-y)\log(1-p)]$$

3. Categorical Cross-Entropy – commonly used for mutually exclusive multi-class classification:

$$L = -\sum_i y_i \log(p_i)$$

4. Sparse Categorical Cross-Entropy – used when multi-class labels are represented by integer class indices rather than

one-hot vectors.

5. Huber loss – combines properties of MSE and absolute error and can be more robust to outliers than MSE.

Example: If a regression model predicts $\hat{y} = 8$ while the actual value is $y = 10$, the squared error is:
 $(10 - 8)^2 = 4$.

A suitable loss function depends on the learning task and output representation. Loss is the training objective, while evaluation metrics such as accuracy or F1-score may be used to assess model performance.

QUESTION 11

What is the purpose of validation and test datasets, and how can they be illustrated with a suitable example?

ANSWER

A dataset is often divided into training, validation, and test sets so that model development and final evaluation remain separated.

1. Training dataset – used to learn model parameters such as weights.
2. Validation dataset – used during development to select models, hyperparameters, architectures, thresholds, or training choices. It should not be treated as the final unbiased evaluation set.
3. Test dataset – kept separate until the final evaluation and used to estimate performance on unseen data.

Example:

Suppose there are 10,000 labeled images:

Training: 7,000 images

Validation: 1,500 images

Test: 1,500 images

Workflow:

All labeled data

↓

Train → → → learn weights

Validation → tune/select model

Test → → → final evaluation

If a model achieves 96% training accuracy but only 80% validation accuracy, overfitting may be occurring. Hyperparameters can be adjusted using the validation set. After the final model is selected, the test set is used once for the

final reported evaluation.

The key purpose is to prevent the test set from becoming part of repeated model-selection decisions.

QUESTION 12

What is padding in a CNN, what is stride, and how do they differ?

ANSWER

Padding is the addition of extra values, usually zeros, around the border of an input before convolution. It controls the spatial size of the feature map and allows border pixels to participate more fully in convolution.

Stride is the number of pixels by which the convolution filter moves at each step.

For a one-dimensional dimension, a common output-size formula is:

$$\text{Output} = \text{floor}((N + 2P - F) / S) + 1$$

where N is input size, P is padding, F is filter size, and S is stride.

Types of padding:

- Valid – no padding.
- Same – padding is selected so that, for stride 1, the output spatial size is generally the same as the input.

Difference:

- Padding changes the effective input boundary by adding values around it.
- Stride controls how far the filter moves.
- Increasing padding can preserve spatial dimensions; increasing stride generally reduces spatial dimensions.

Example: with a 5×5 input, 3×3 filter, padding P=1 and stride S=1:

$$\text{Output} = (5 + 2 - 3) / 1 + 1 = 5.$$

Thus the output remains 5×5.

QUESTION 13

What is transfer learning, and what are the advantages of using them?

ANSWER

Transfer learning is a technique in which knowledge learned by a model on one task or dataset is reused for a related

target task. In deep learning, a pre-trained model is often used as a starting point instead of training an entire network from

scratch.

Example:

A CNN pre-trained on a large image dataset may have learned useful low-level and mid-level visual features.

For a smaller

flower-classification dataset, the final classification layer can be replaced with a new layer for the flower classes.

Typical process:

Pre-trained model → Replace/adapt output layer → Train on target data

Fine-tuning is a related approach in which some previously trained layers are unfrozen and trained with the target dataset,

usually using a small learning rate.

Advantages:

1. Requires less target-task training data.
2. Reduces training time and computational cost.
3. Often improves performance when the target task is related to the source task.
4. Provides useful learned representations.
5. Can reduce the difficulty of training deep models from random initialization.
6. Particularly useful when labeled target data is limited.

Transfer learning is most effective when the source and target domains/tasks have useful similarities.

QUESTION 14

What is the architecture of a Gated Recurrent Unit (GRU), and what are its main gates?

ANSWER

A Gated Recurrent Unit (GRU) is a gated recurrent architecture that controls information flow using two main gates: the

update gate and the reset gate.

1. Update gate:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

It controls how much of the previous hidden state is retained and how much new candidate information is incorporated.

2. Reset gate:

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

It controls how strongly the previous hidden state contributes when computing the candidate state.

3. Candidate hidden state:

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h)$$

4. Final hidden state:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Here σ is sigmoid and $\odot$ denotes element-wise multiplication.

Simple architecture:

$$x_t + h_{t-1}$$

↓

Reset gate →

Update gate → Candidate/State computation → h_t

Unlike an LSTM, a standard GRU does not maintain a separate cell state. Its gating mechanism uses the hidden state to store and control information.

QUESTION 15

What are the important factors that affect the outcome of an NLP model, and can you illustrate them with suitable examples?

ANSWER

The outcome of an NLP model depends on the quality of data, representation, preprocessing, model design, and evaluation.

Important factors include:

1. Data quality and quantity – noisy or insufficient training data can reduce generalization. Example: duplicate or mislabeled reviews can mislead a sentiment classifier.
2. Data representativeness – training text should reflect the target domain. A general-language dataset may not adequately represent medical terminology.
3. Preprocessing – inappropriate tokenization or removal of useful words can harm performance.
4. Feature representation – TF-IDF, word embeddings, and contextual embeddings represent language differently.
5. Vocabulary and out-of-vocabulary handling – rare or unseen words may reduce performance.
6. Class imbalance – if 95% of examples belong to one class, accuracy alone may hide poor minority-class performance.
7. Model architecture – CNNs, RNNs, LSTMs, Transformers, or other models have different strengths.
8. Hyperparameters – learning rate, batch size, number of layers, and regularization affect training.
9. Overfitting/underfitting – a model may memorize training data or fail to learn enough.
10. Evaluation design – suitable metrics and representative test data are essential.

Example: For spam detection, using representative emails, correct labels, suitable tokenization, a useful text representation, balanced evaluation, and appropriate hyperparameters can all affect the final precision and recall.

QUESTION 16

Why is model optimization needed in an NLP application, and what are suitable optimization approaches with examples?

ANSWER

Model optimization is needed to improve an NLP model's ability to learn useful patterns, generalize to unseen text, and train efficiently. Optimization can refer both to minimizing the training objective and to practical improvements in model performance and efficiency.

For parameter learning, gradient-based methods calculate gradients through backpropagation and update parameters using an optimizer.

Basic gradient descent:

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

where θ represents model parameters, η is the learning rate, and L is the loss.

Suitable approaches include:

1. Stochastic Gradient Descent (SGD) – updates parameters using individual examples.
2. Mini-batch Gradient Descent – uses a small batch of examples and is widely used in deep learning.
3. Momentum – uses a moving direction based on previous gradients to accelerate learning.
4. RMSProp – adapts the learning rate using a moving average of squared gradients.
5. Adam – combines adaptive learning-rate ideas with momentum-like first-moment estimates and is widely used for NLP models.
6. Learning-rate scheduling – changes the learning rate during training.
7. Regularization/dropout – reduces overfitting.
8. Early stopping – stops training when validation performance stops improving.

Example: For sentiment classification, Adam with an appropriate learning rate, mini-batches, dropout, and early stopping

may provide stable training and better validation generalization than an unsuitable learning setup.

PART B

15-MARK QUESTIONS

QUESTION 1

How can different optimization algorithms used in deep learning be categorized and explained?

ANSWER

Optimization algorithms determine how neural-network parameters are updated to minimize a loss function. They can be

categorized according to how gradients and learning rates are used.

1. Gradient-descent family

Batch Gradient Descent computes the gradient using the complete training set:

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

It can be stable but may be computationally expensive for large datasets.

2. Stochastic Gradient Descent (SGD)

SGD updates parameters using one training example at a time:

$$\theta_{t+1} = \theta_t - \eta \nabla L_i(\theta_t)$$

It is computationally inexpensive per update but can produce noisy updates.

3. Mini-batch Gradient Descent

A mini-batch of m examples is used:

$$\theta_{t+1} = \theta_t - \eta (1/m) \sum_{i \in B} \nabla L_i(\theta_t)$$

It provides a practical balance between computational efficiency and gradient stability and is the standard approach in

many deep-learning systems.

4. Momentum-based methods

Momentum maintains a velocity:

$$v_t = \beta v_{t-1} + (1-\beta) \nabla L(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

It can accelerate movement in useful directions and reduce oscillation.

5. Adaptive learning-rate methods

RMSProp maintains a moving average of squared gradients and scales updates accordingly.

6. Adam

Adam combines first-moment and second-moment estimates:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

Bias-corrected estimates are then used for the parameter update.

Other techniques include learning-rate schedules, AdamW, and adaptive methods designed for particular training requirements.

Important distinction:

Backpropagation computes gradients. The optimizer uses those gradients to update parameters. They are complementary, not competing descriptions of the same operation.

Selection depends on dataset size, model architecture, stability, memory, computational cost, and generalization behaviour.

QUESTION 2

How do Perceptron and Multilayer Perceptron (MLP) differ, and how can they be explained with suitable examples, including MLP architecture and applications?

ANSWER

A Perceptron is a single artificial neuron, whereas a Multilayer Perceptron (MLP) is a feed-forward neural network containing one or more hidden layers.

Perceptron:

$x_1, x_2, \dots, x_n \rightarrow \text{weighted sum} + \text{bias} \rightarrow \text{activation} \rightarrow \text{output}$

$$z = \sum w_i x_i + b$$

$$y = f(z)$$

A single Perceptron can learn linearly separable binary decision boundaries. It cannot solve a non-linearly separable problem such as XOR.

MLP architecture:

Input Layer → Hidden Layer 1 → Hidden Layer 2 → Output Layer

x [neurons] [neurons] y

↓ activation functions

Each hidden neuron performs:

$$z = Wx + b$$

$$a = f(z)$$

The final layer uses an activation appropriate to the task, such as sigmoid for binary classification or Softmax for mutually exclusive multi-class classification.

Example:

For XOR, a single Perceptron fails because no single straight-line decision boundary separates the classes. An MLP with a hidden layer can learn the required non-linear decision function.

Key differences:

- Structure: one neuron vs. multiple layers of neurons.
- Representation: Perceptron learns a linear boundary; MLP can learn non-linear relationships.
- Training: MLP normally uses backpropagation with gradient-based optimization.
- Capacity: MLP has substantially greater representational capacity.

Applications of MLPs include tabular classification, regression, pattern recognition, risk prediction, customer classification, and function approximation.

Advantages of MLP: learns non-linear mappings and supports many input/output configurations.

Limitations: can overfit, may require careful hyperparameter tuning, and is usually less naturally suited than CNNs or sequence models to raw images or sequential data.

QUESTION 3

What are the major regularization methods used to overcome overfitting, and how can they be explained with a suitable example?

ANSWER

Overfitting occurs when a model learns training-specific patterns or noise and performs substantially worse on unseen

data. Regularization reduces this problem by constraining the model or training process.

1. L1 regularization

Adds the absolute values of weights to the objective:

$$L_{\text{total}} = L_{\text{data}} + \lambda \sum |w_i|$$

It can encourage some weights toward zero and therefore produce sparse models.

2. L2 regularization

Adds squared weights:

$$L_{\text{total}} = L_{\text{data}} + \lambda \sum w_i^2$$

It discourages excessively large weights and is widely used in neural networks.

3. Dropout

During training, randomly selected neuron outputs are temporarily set to zero. This prevents the network from relying too

strongly on particular units. At inference, dropout is disabled and the appropriate scaling is handled by the framework.

4. Early stopping

Training is stopped when validation performance stops improving or begins to deteriorate.

5. Data augmentation

For images, transformations such as small rotations, translations, crops, or flips can create varied training examples while

preserving the class.

6. Reducing model complexity

Fewer layers/units can reduce the model's capacity when the original network is unnecessarily large.

7. Transfer learning

Using a pre-trained representation can be useful when the target dataset is small, although it is not itself a regularizer in

the strict mathematical sense.

Example:

Suppose a CNN reaches 99% training accuracy but 82% validation accuracy. Adding dropout, using suitable weight

regularization, augmenting the training images, and applying early stopping can reduce the gap.

Regularization must be selected carefully: excessive regularization can cause underfitting. The validation set is used to

select suitable regularization strength.

QUESTION 4

What are the various functions of optimizers and optimization techniques used in neural network learning, and what are

their applications?

ANSWER

An optimizer controls how trainable parameters are updated after gradients have been calculated. Its main purpose is to

minimize the training loss efficiently.

General update:

$$\theta_{t+1} = \theta_t - \eta \cdot \text{Update}(g_t)$$

where θ is a parameter, η is the learning rate, and g_t is its gradient.

Major optimizer functions and techniques:

1. SGD – simple gradient-based updates; useful as a baseline and in many large-scale models.
2. Mini-batch SGD – computes updates from batches, making efficient use of vectorized hardware.
3. Momentum – accumulates a moving direction, accelerating learning in consistent directions and reducing oscillations.
4. Nesterov momentum – evaluates the gradient using a look-ahead position.
5. AdaGrad – adapts learning rates using accumulated squared gradients; useful when features have different frequencies, although learning rates can become very small.
6. RMSProp – uses an exponentially weighted average of squared gradients and is useful for non-stationary objectives.

7. Adam – combines adaptive second-moment scaling with first-moment estimates and is widely used for deep-learning tasks.
8. AdamW – separates weight decay from the adaptive gradient update and is commonly used in modern neural-network training.
9. Learning-rate scheduling – changes η during training, for example by decay or cosine schedules.

Applications include image classification, NLP, speech models, regression networks, and large neural architectures.

Optimization workflow:

Forward pass → Loss calculation → Backpropagation → Gradients → Optimizer update → Repeat.

An optimizer does not replace backpropagation. Backpropagation supplies the gradients; the optimizer decides how those gradients are converted into parameter updates.

QUESTION 5

How do LeNet, AlexNet, VGG, ResNet, and Inception architectures differ, and what are the advantages of each architecture?

ANSWER

These CNN architectures represent important stages in the development of deep image-recognition networks.

LeNet:

Typical structure: convolution → pooling → convolution → pooling → fully connected layers.

It was designed for handwritten-digit recognition.

Advantage: simple, computationally efficient, and historically important for demonstrating CNN-based visual recognition.

AlexNet:

It used a substantially deeper/wider CNN than LeNet and popularized deep CNNs for large-scale image recognition.

Important design choices included ReLU activations, pooling, dropout, and GPU-based training.

Advantage: much stronger feature learning for large image datasets and a landmark improvement in image classification.

VGG:

VGG uses many small 3×3 convolution filters in a relatively uniform stacked architecture, followed by pooling and

classification layers.

Advantage: simple, regular, and easy-to-understand architecture; its repeated blocks made depth systematic.

Limitation: large parameter/computational requirements in the original versions.

Inception:

Inception modules process information with multiple convolutional operations of different effective receptive-field sizes in

parallel and combine their outputs. 1×1 convolutions are used in the family to control computation and channel dimensions.

Advantage: multi-scale feature extraction with improved computational efficiency relative to simply making every layer

wider.

ResNet:

ResNet introduces residual/skip connections:

$$y = F(x) + x$$

A block learns a residual mapping $F(x)$ while the identity path carries x .

Advantage: enables very deep networks to train more effectively and helps mitigate degradation/optimization difficulties

associated with simply stacking many layers.

Comparison:

LeNet — early compact CNN.

AlexNet — large-scale deep CNN breakthrough.

VGG — deep, uniform 3×3 convolution stacks.

Inception — parallel multi-scale processing.

ResNet — residual/skip connections for very deep networks.

All are CNN families, but they differ in depth, connectivity, computational design, and feature-extraction strategy.

QUESTION 6

What is the architecture of a Convolutional Neural Network (CNN), and how can it be explained with a net diagram and a suitable example?

ANSWER

A Convolutional Neural Network is a deep neural architecture designed especially for grid-like data such as images. It

learns spatial features through local receptive fields and shared convolution filters.

Typical architecture:

Input Image

↓

Convolution → Activation

↓

Pooling

↓

Convolution → Activation

↓

Pooling

↓

Flatten / Global Average Pooling

↓

Fully Connected Layer(s)

↓

Output Layer

Convolution:

A filter slides across the input and produces a feature map. Multiple filters learn different features.

Activation:

ReLU is commonly used:

$\text{ReLU}(x) = \max(0, x)$

It introduces non-linearity.

Pooling:

Pooling reduces spatial dimensions and can make representations less sensitive to small translations. Max pooling selects the maximum value in a local window.

Flatten:

Converts a feature tensor into a vector for dense classification layers. Modern CNNs may instead use global average pooling.

Output:

For binary classification, a sigmoid output is common. For mutually exclusive multi-class classification, Softmax is commonly used.

Example: Cat vs. dog classification.

Input: resized RGB image

- convolution layers learn edges and textures
- deeper layers learn shapes and object parts
- pooling reduces spatial size
- dense/output layer produces class probabilities.

CNN advantages include local connectivity, parameter sharing, hierarchical feature learning, and strong performance on

image tasks. Important design factors include filter size, number of filters, padding, stride, pooling, activation, depth, and regularization.

QUESTION 7

What is the architecture of a Recurrent Neural Network (RNN), and what are its applications and limitations?

ANSWER

An RNN is designed for sequential data. It processes one time step at a time while maintaining a hidden state that carries information from previous steps.

At time t :

$$h_t = f(W_{xh} x_t + W_{hh} h_{t-1} + b_h)$$

$$y_t = g(W_{hy} h_t + b_y)$$

Architecture:

$x_t \rightarrow [\text{RNN}] \rightarrow h_t \rightarrow y_t$

↑

$x_t \rightarrow [\text{RNN}] \rightarrow h_t \rightarrow y_t$

↑

$x_t \rightarrow [\text{RNN}] \rightarrow h_t \rightarrow y_t$

↑

...

The same recurrent parameters are shared across time steps.

Applications:

- NLP and text processing
- Language modeling and text generation
- Speech recognition
- Machine translation
- Time-series forecasting
- Handwriting recognition
- Video/activity recognition

Limitations:

1. Vanishing gradients can make learning long-range dependencies difficult.
2. Exploding gradients can make training unstable.
3. Sequential computation reduces parallelism across time steps.
4. Long sequences can be difficult for a basic RNN to remember.
5. Training can be slower than highly parallel architectures for suitable tasks.

LSTM and GRU address some of these limitations using gating mechanisms.

Example:

For a sentence, the RNN processes words/tokens sequentially and updates h_t so that information from earlier words can

influence later predictions.

QUESTION 8

What is the architecture of a Gated Recurrent Unit (GRU), and what are its suitable applications?

ANSWER

A GRU is a gated recurrent neural network architecture that controls information flow using two main gates: the update

gate and reset gate. It has no separate cell state like an LSTM.

Update gate:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

Reset gate:

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

Candidate:

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h)$$

Final state:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Architecture:

$x_t \rightarrow \rightarrow \rightarrow$

$h_{t-1} \rightarrow \rightarrow \rightarrow$ Update gate $\rightarrow \rightarrow \rightarrow \rightarrow$

$\rightarrow$ Reset gate $\rightarrow$ Candidate $\rightarrow$ combine $\rightarrow h_t$

$\rightarrow \rightarrow$

The update gate determines the balance between retaining the previous state and incorporating the candidate state. The

reset gate controls the influence of the previous state when forming the candidate.

Suitable applications include:

- Text classification
- Sentiment analysis
- Sequence prediction
- Speech-related sequence modeling
- Time-series forecasting
- Language modeling
- Other sequential data where gated memory is useful.

Advantages include fewer gate/state components than a standard LSTM and often efficient training. A GRU can perform

well on many sequence tasks, but the best architecture depends on the data and application.

QUESTION 9

How can an NLP model engineering pipeline be designed for a text classification application, including model evaluation and applications?

ANSWER

An NLP text-classification pipeline should move systematically from problem definition and data collection to preprocessing, representation, modeling, evaluation, and deployment.

Pipeline:

Problem definition

↓

Data collection and labeling

↓

Data inspection and cleaning

↓

Train/validation/test split

↓

Tokenization and normalization

↓

Text representation

↓

Model selection

↓

Training + validation

↓

Hyperparameter tuning

↓

Test evaluation

↓

Error analysis

↓

Deployment and monitoring

1. Define the task

Specify classes, input, output, and success metrics. Example: classify customer reviews as positive, negative, or neutral.

2. Prepare data

Collect representative text and correct labels. Remove duplicates and handle missing/corrupted records.

3. Preprocess

Depending on the model, perform tokenization, normalization, punctuation handling, and appropriate handling of URLs or

special tokens. Avoid preprocessing that destroys information useful to the selected model.

4. Represent text

Options include Bag-of-Words, TF-IDF, word embeddings, and contextual representations.

5. Select a model

A baseline can be logistic regression or another classifier over TF-IDF. Deep-learning alternatives include CNNs,

RNN/LSTM/GRU models, and Transformer-based models.

6. Train and tune

Use a validation set for hyperparameters such as learning rate, batch size, regularization, architecture, and threshold.

7. Evaluate

Use a held-out test set. For classification, report accuracy where appropriate and also precision, recall, F1-score, and a

confusion matrix when class distribution or error types require them.

8. Error analysis

Inspect false positives/negatives, class imbalance, ambiguous language, domain-specific terms, and data-quality issues.

Applications include spam detection, sentiment analysis, topic classification, intent detection, support-ticket routing, and

document categorization.

QUESTION 10

What factors affect NLP performance, and what suitable optimization techniques can be used to improve the model

output?

ANSWER

NLP performance is influenced by data, preprocessing, representation, architecture, optimization, and evaluation.

Major factors:

1. Data quantity and quality.
2. Correct and consistent labels.
3. Domain representativeness.
4. Class imbalance.
5. Tokenization and preprocessing choices.
6. Vocabulary coverage and handling of rare/unseen words.
7. Choice of representation such as TF-IDF, embeddings, or contextual representations.
8. Model architecture and capacity.
9. Hyperparameters.
10. Overfitting and underfitting.
11. Evaluation metric and test-set design.
12. Computational resources and training duration.

Optimization approaches:

- Mini-batch training.
- SGD, Momentum, RMSProp, Adam/AdamW.
- Learning-rate scheduling and warm-up where appropriate.
- Dropout and weight decay.

- Early stopping.
- Data augmentation or task-appropriate text perturbation where label meaning is preserved.
- Class weighting or resampling for imbalance.
- Hyperparameter search.
- Transfer learning and fine-tuning.
- Threshold tuning for classifiers when the application requires a precision/recall trade-off.

Workflow:

Data quality → preprocessing → representation → baseline → optimizer/hyperparameters → validation → error analysis → refinement → final test.

Example: In an imbalanced spam classifier, accuracy may be misleading. Class-aware metrics, class weighting, a suitable

threshold, and error analysis can improve the practical quality of predictions.

Optimization should be evidence-driven using validation data rather than repeated tuning against the test set.

QUESTION 11

How can a deep neural network be designed using TensorFlow/Keras for a simple classification task, including its architecture, implementation, and output?

ANSWER

A simple classification network can be designed in TensorFlow/Keras by defining the input, hidden layers, output layer, loss, optimizer, and metrics.

Example task: classify tabular samples into two classes.

Architecture:

Input features

↓

Dense(64, ReLU)

↓

Dropout

↓
Dense(32, ReLU)
↓
Dense(1, Sigmoid)
↓
Binary class probability

Representative implementation:

```
from tensorflow import keras  
from tensorflow.keras import layers
```

```
model = keras.Sequential([  
    layers.Input(shape=(n_features,)),  
    layers.Dense(64, activation="relu"),  
    layers.Dropout(0.3),  
    layers.Dense(32, activation="relu"),  
    layers.Dense(1, activation="sigmoid")  
])
```

```
model.compile(  
    optimizer="adam",  
    loss="binary_crossentropy",  
    metrics=["accuracy"]  
)
```

```
model.fit(  
    X_train, y_train,  
    validation_data=(X_val, y_val),  
    epochs=20,  
    batch_size=32  
)
```

```
pred = model.predict(X_test)
```

The output of the sigmoid layer is between 0 and 1 and can be converted into a class using an application-selected

threshold, commonly 0.5 for a simple demonstration.

For multi-class classification, the output layer can instead use:

`Dense(number_of_classes, activation="softmax")`

with an appropriate categorical cross-entropy loss.

A complete implementation should include preprocessing, a separate test set, validation monitoring, suitable metrics, and evaluation after training.

QUESTION 12

How do forward propagation, backpropagation, and gradient descent work in training neural networks, including their algorithms, equations, and workflow?

ANSWER

Training a neural network involves three connected stages: forward propagation computes predictions, backpropagation computes gradients of the loss with respect to parameters, and gradient-based optimization updates the parameters.

1. Forward propagation

For a layer:

$$z^l = W^l a^{l-1} + b^l$$

$$a^l = f(z^l)$$

The process starts with input $a = x$ and continues to the output. The prediction is compared with the target using a loss

function L .

2. Backpropagation

Backpropagation applies the chain rule to calculate gradients efficiently.

For an output layer:

$$\delta^l = \partial L / \partial z^l$$

For a hidden layer:

$$\delta^l = (W^{l+1})^T \delta^{l+1} \odot f'(z^l)$$

Gradients:

$$\partial L / \partial W^l = \delta^l (a^{l-1})^T$$

$$\partial L / \partial b^l = \delta^l$$

3. Gradient descent

Parameters are updated using:

$$W \leftarrow W - \eta (\partial L / \partial W)$$

$$b \leftarrow b - \eta (\partial L / \partial b)$$

Workflow:

Input $\rightarrow$ Forward pass $\rightarrow$ Prediction $\rightarrow$ Loss

$\downarrow$

Backpropagation

$\downarrow$

Gradients

$\downarrow$

Optimizer/update

$\downarrow$

Next iteration

Algorithm:

1. Initialize weights and biases.
2. Select a mini-batch.
3. Perform forward propagation.
4. Compute loss.
5. Propagate error backward using the chain rule.
6. Calculate parameter gradients.
7. Apply an optimization update.
8. Repeat over batches and epochs.
9. Monitor validation performance.
10. Stop when the training procedure reaches an appropriate stopping condition.

Backpropagation is gradient computation; gradient descent/SGD/Adam is parameter optimization using those gradients.

QUESTION 13

What are the methods used for data preprocessing, feature engineering, and hyperparameter tuning, and how can they be illustrated with an example and workflow?

ANSWER

Data preprocessing prepares raw data, feature engineering creates useful representations, and hyperparameter tuning selects training/design settings that improve validation performance.

1. Data preprocessing

- Handle missing values.
- Remove duplicates and obvious corrupt records.
- Detect or appropriately handle outliers.
- Encode categorical variables.
- Scale numerical variables where required.
- Split into training, validation, and test sets.
- For text: tokenize and normalize appropriately.
- For images: resize and normalize pixel values.

2. Feature engineering

- Feature extraction from dates, text, images, or domain measurements.
- One-hot or other suitable categorical representations.
- Interaction/derived features.
- TF-IDF or embeddings for text.
- Normalized or transformed numerical features.

3. Hyperparameter tuning

Hyperparameters are selected before/during training rather than learned as ordinary model weights. Examples:

- Learning rate.
- Batch size.
- Number of layers/units.
- Dropout rate.
- Regularization strength.

- Number of trees or other model-specific settings.

Methods:

- Manual search.
- Grid search.
- Random search.
- Bayesian/optimization-based search.
- Early-pruning approaches when supported.

Example workflow for customer classification:

Raw data

- clean missing/duplicate values
- split train/validation/test
- encode categories + scale numeric features
- create derived features
- train baseline
- tune learning rate/dropout/hidden units using validation
- select best configuration
- retrain according to the chosen protocol
- evaluate once on the test set.

A critical principle is to prevent data leakage: preprocessing parameters such as scaling statistics must be learned from

training data and then applied to validation/test data.

QUESTION 14

How does backpropagation work with stochastic gradient descent and mini-batch gradient descent, including their

algorithms, equations, and parameters?

ANSWER

Backpropagation computes gradients of the loss with respect to network parameters. SGD and mini-batch gradient

descent determine which training examples are used to form each parameter update.

For parameters θ and per-example loss L :

$$g_i = \nabla \theta L_i(\theta)$$

Stochastic Gradient Descent:

$$\theta_{t+1} = \theta_t - \eta g_t$$

Here one example is used for each update.

Mini-batch Gradient Descent:

For batch B of size m:

$$g_B = (1/m) \sum_{i \in B} \nabla \theta L_i(\theta)$$

$$\theta_{t+1} = \theta_t - \eta g_B$$

Algorithm for either method:

1. Initialize θ .
2. Shuffle the training data where appropriate.
3. Select one example (SGD) or a mini-batch (mini-batch GD).
4. Forward propagate to obtain predictions.
5. Compute the loss.
6. Backpropagate the loss to calculate gradients.
7. Update parameters using the gradient and learning rate.
8. Repeat for all examples/batches.
9. Repeat for multiple epochs.
10. Monitor validation performance.

Important parameters:

- Learning rate η .
- Batch size m for mini-batch training.
- Number of epochs.
- Model parameters θ .
- Loss function.
- Optional momentum or adaptive-optimizer parameters.

Comparison:

SGD: frequent, noisy updates; low memory per update.

Mini-batch: less noisy and highly compatible with vectorized GPU computation; batch size controls the trade-off.

Backpropagation is the same gradient-calculation principle in both cases; only the amount of data used to estimate each update gradient changes.

QUESTION 15

What are transfer learning and fine-tuning using pre-trained models for image classification, and how can they be demonstrated with a suitable example?

ANSWER

Transfer learning reuses representations learned from a source task for a related target task. Fine-tuning is a transfer-learning strategy in which selected pre-trained layers are subsequently unfrozen and trained on the target dataset.

Example: classifying different types of flowers with a relatively small labeled dataset.

Architecture:

Pre-trained CNN backbone → learned visual features → new target classifier

↓

flower class probabilities

Step 1: Choose a pre-trained model

For example, use a CNN pre-trained on a large image dataset.

Step 2: Adapt the input and output

Keep the compatible feature-extraction layers and replace the original classification head with one matching the target classes.

Step 3: Freeze the backbone

Initially prevent the pre-trained layers from changing and train the new classification head.

Step 4: Fine-tune

Unfreeze some upper backbone layers and train them with a small learning rate so that general features can adapt to the target domain.

Step 5: Evaluate

Use validation data for model choices and a held-out test set for final performance.

Conceptual Keras example:

```
base = keras.applications.MobileNetV2(
    weights="imagenet", include_top=False
)
base.trainable = False

model = keras.Sequential([
    base,
    layers.GlobalAveragePooling2D(),
    layers.Dense(num_classes, activation="softmax")
])
```

After initial training, selected base layers can be unfrozen and the model recompiled with a small learning rate for fine-tuning.

Advantages:

- Less labeled target data may be required.
- Training can be faster.
- Pre-trained features can improve generalization.
- Fine-tuning adapts the representation to the target domain.

Care is required when source and target domains differ strongly, because transferred features may not be equally useful.

QUESTION 16

What are the evaluation metrics used for CNNs, and how does data augmentation improve model performance with a suitable example?

ANSWER

CNN evaluation metrics depend on the task.

For classification:

1. Accuracy:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Useful when class distributions and error costs make overall correctness informative.

2. Precision:

$$\text{Precision} = TP / (TP + FP)$$

Measures how many predicted positives are actually positive.

3. Recall:

$$\text{Recall} = TP / (TP + FN)$$

Measures how many actual positives are detected.

4. F1-score:

$$F1 = 2(\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

5. Confusion matrix

Shows counts of TP, TN, FP, and FN and is useful for understanding class-specific errors.

6. Multi-class metrics

Precision, recall, and F1 can be reported using suitable macro, weighted, or micro averaging depending on the evaluation objective.

For regression-like CNN outputs, metrics such as MAE or RMSE may be appropriate.

Data augmentation creates additional varied training examples through label-preserving transformations. For image

classification, examples include small rotations, translations, crops, flips where semantically valid, brightness changes, or

other task-appropriate transformations.

Example:

A plant-image classifier has only 2,000 training images. Augmentation can expose the CNN to varied orientations and crops, reducing reliance on exact training-image appearance.

Workflow:

Original training images → augmentation → varied training batches → CNN → validation/test evaluation.

Augmentation is normally applied to training data, not as uncontrolled transformations of the final test set. It can reduce

overfitting and improve robustness, but transformations must preserve the correct label.

QUESTION 17

What is the architecture of an LSTM, and what are the functions of its input, forget, and output gates?

ANSWER

An LSTM is a recurrent neural architecture designed to learn dependencies over longer sequences. It maintains a cell

state C_t and a hidden state h_t .

Inputs at time t are x_t and h_{t-1} .

Forget gate:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

It determines which parts of the previous cell state $C_t \rightarrow$ should be retained.

Input gate:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

It controls how much new information enters the cell.

Candidate state:

$$\rightarrow = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

Cell update:

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \rightarrow$$

Output gate:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

It determines how much of the updated cell state is exposed through the hidden state.

Hidden state:

$$h_t = o_t \tanh(C_t)$$

Architecture:

→ Forget gate →

x_t, h_{t-1} →→ Input gate →→→ Cell state C_t → Output gate → h_t

→ Candidate →→→

Functions:

- Forget gate: discard unnecessary old information.
- Input gate: control new information written to memory.
- Candidate: proposes new content.
- Cell state: carries long-term information.
- Output gate: controls the exposed hidden state.

The gating mechanism helps LSTMs handle long-term dependencies more effectively than a basic RNN in many sequence tasks.

QUESTION 18

How do RNN, LSTM, and GRU compare in terms of their architectures, advantages, limitations, and suitable examples?

ANSWER

RNN, LSTM, and GRU are recurrent architectures for sequential data, but they differ in how they maintain and control state.

RNN:

$$h_t = f(W_{xx_t} + W_{hh_{t-1}} + b)$$

Architecture: one hidden state passed through time.

Advantages: simple and relatively lightweight.

Limitations: vanishing/exploding gradients and difficulty retaining long-range information.

Example: simple short-sequence prediction.

LSTM:

Maintains hidden state h_t and separate cell state C_t using forget, input, and output gates.

Advantages: strong mechanism for controlling long-term information.

Limitations: more parameters and computational complexity than a basic RNN; sequential computation remains.

Example: long-context sequence modeling or time-series tasks where gated memory is useful.

GRU:

Uses update and reset gates and one main hidden state.

Advantages: simpler than LSTM, fewer state components, and often efficient while retaining gated-memory benefits.

Limitations: does not provide a separate LSTM-style cell state; performance depends on task.

Example: text classification or time-series prediction.

Comparison:

Feature | RNN | LSTM | GRU

State | hidden | hidden + cell | hidden

Gates | none in basic RNN | forget, input, output | update, reset

Long dependencies | difficult | generally better | generally better

Complexity | lowest | highest of the three | intermediate

Use | simpler sequences | complex/long dependencies | efficient gated sequence modeling

All three process sequences recurrently. The suitable choice depends on dataset, sequence length, computational

constraints, and validation performance.

QUESTION 19

How can a suitable NLP solution be developed by selecting appropriate packages, defining data requirements, engineering

the model, and evaluating performance?

ANSWER

A suitable NLP solution begins with a clearly defined problem and ends with validated performance and error analysis.

1. Define the problem

Specify input, output classes, users, constraints, and success metrics. Example: classify support messages into billing, technical, account, and other categories.

2. Define data requirements

Collect representative text and labels. Require sufficient volume, correct annotations, domain coverage, manageable noise, and appropriate train/validation/test separation.

3. Select packages

- pandas – data loading and cleaning.
- NumPy – numerical processing.
- NLTK/spaCy – tokenization and linguistic processing.
- scikit-learn – TF-IDF and classical models.
- TensorFlow/Keras or PyTorch – deep-learning models.

4. Prepare data

Clean duplicates and corrupted records, handle missing values, tokenize/normalize as appropriate, and prevent leakage.

5. Engineer the representation/model

Baseline:

Text → TF-IDF → classifier.

Deep-learning alternatives:

Text → tokenization/vectorization → embedding → CNN/RNN/LSTM/GRU/other neural architecture → output.

6. Train

Use a suitable loss and optimizer, validation data, and regularization where needed.

7. Evaluate

Use accuracy when appropriate and precision, recall, F1-score, confusion matrix, and class-specific analysis when they provide better information.

8. Error analysis

Inspect misclassified examples to identify ambiguous language, missing vocabulary, mislabeled records, imbalance, or preprocessing errors.

9. Deployment

Integrate the model, monitor input drift and performance, and retrain when justified by new data.

The complete workflow is:

Problem → Data → Preprocessing → Representation → Model → Training → Validation → Test → Error analysis →

Deployment/monitoring.

QUESTION 20

How can an NLP model development process be evaluated, and how can an appropriate optimization strategy be formulated?

ANSWER

NLP model development should be evaluated as a complete process rather than by training accuracy alone.

Evaluation stages:

1. Problem and metric definition – decide what constitutes useful performance.
2. Data audit – assess volume, quality, label consistency, domain coverage, duplicates, and class balance.
3. Split integrity – maintain separate training, validation, and test sets and prevent leakage.
4. Baseline – establish a simple reference model.
5. Training monitoring – track training and validation loss/metrics to identify overfitting or underfitting.
6. Final test evaluation – after model-selection decisions are complete, evaluate on the held-out test set.
7. Error analysis – inspect false positives, false negatives, class-specific errors, and difficult examples.
8. Robustness/generalization – assess performance on representative variations or relevant subgroups where appropriate.

Optimization strategy formulation:

Step 1: Establish baseline.

Step 2: Identify bottleneck from validation/error analysis.

Step 3: Select an intervention.

Step 4: Change one or a controlled set of factors.

Step 5: Compare on validation data.

Step 6: Retain the change only if it improves the chosen objective without unacceptable trade-offs.

Step 7: Repeat systematically.

Step 8: Freeze the final configuration and evaluate once on the test set.

Possible optimization choices:

- Adam/AdamW or SGD variants.
- Learning-rate tuning and scheduling.
- Batch-size tuning.
- Dropout and weight decay.
- Class weighting for imbalance.
- Better preprocessing/representation.
- Transfer learning/fine-tuning.
- Hyperparameter search.
- Early stopping.

A sound strategy links each optimization change to observed evidence. The test set should not be repeatedly used to

select improvements because that can produce an overly optimistic estimate of generalization.

FINAL REVISION CHECKLIST

IMPORTANT ARCHITECTURES TO MEMORIZE

ANN: Input $\rightarrow$ Hidden layer(s) $\rightarrow$ Output.

CNN: Input $\rightarrow$ Convolution $\rightarrow$ Activation $\rightarrow$ Pooling $\rightarrow$ repeated feature extraction $\rightarrow$ Flatten/Global Average Pooling $\rightarrow$

Dense $\rightarrow$ Output.

RNN: $x_t + h_{t-1} \rightarrow$ recurrent cell $\rightarrow h_t \rightarrow$ output.

LSTM: Forget gate + Input gate + Candidate $\rightarrow$ Cell-state update $\rightarrow$ Output gate $\rightarrow h_t$.

GRU: Update gate + Reset gate + Candidate $\rightarrow h_t$.

MLP: Input $\rightarrow$ Dense hidden layer(s) $\rightarrow$ Output.

Residual block: $F(x) + x$.

Inception: parallel branches with different operations $\rightarrow$ concatenation.

IMPORTANT EQUATIONS TO MEMORIZE

Neuron: $z = \sum w_i x_i + b$.

Sigmoid: $\sigma(z) = 1/(1+e^{-z})$.

Softmax: $\text{softmax}(z_i) = e^{-z_i} / \sum e^{-z_j}$.

ReLU: $\max(0, x)$.

MSE: $(1/n) \sum (y_i - \hat{y}_i)^2$.

Binary cross-entropy: $-\left[y \log(p) + (1-y) \log(1-p)\right]$.

Gradient descent: $\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$.

CNN output size: $\text{floor}((N+2P-F)/S)+1$.

LSTM: $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$; $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$; $C_t = f_t \odot C_{t-1} + i_t \odot \tanh(W_c[h_{t-1}, x_t] + b_c)$; $o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$; $h_t = o_t \tanh(C_t)$.

GRU: $z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$; $r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$; $\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1} + x_t] + b_h)$; $h_t = (1 - z_t) h_{t-1} + z_t \tilde{h}_t$.

IMPORTANT ALGORITHMS

Forward propagation $\rightarrow$ loss calculation $\rightarrow$ backpropagation $\rightarrow$ gradient calculation $\rightarrow$ optimizer update.

SGD: one example per update.

Mini-batch GD: average gradient over a mini-batch.

Adam: first- and second-moment estimates with bias correction.

CNN training: convolution $\rightarrow$ activation $\rightarrow$ pooling/feature extraction $\rightarrow$ classification $\rightarrow$ loss $\rightarrow$ backpropagation $\rightarrow$ update.

NLP pipeline: problem definition $\rightarrow$ data $\rightarrow$ cleaning $\rightarrow$ split $\rightarrow$ tokenization/representation $\rightarrow$ model $\rightarrow$ training $\rightarrow$

validation $\rightarrow$ test $\rightarrow$ error analysis $\rightarrow$ deployment.

IMPORTANT COMPARISONS

Perceptron vs MLP: single neuron/linear boundary vs multiple layers/non-linear representation.

RNN vs LSTM vs GRU: basic recurrent state vs gated cell+hidden state vs two-gate hidden-state architecture.

Sigmoid vs Softmax: independent 0–1 output vs normalized mutually exclusive class distribution.

Padding vs stride: border extension vs filter movement.

Transfer learning vs fine-tuning: reuse of learned representation vs additional training of selected pre-trained layers.

IMPORTANT DIAGRAMS

Practice drawing: ANN layer diagram; Perceptron neuron; CNN pipeline; RNN unfolded through time; LSTM gate/cell

diagram; GRU gate diagram; forward/backpropagation workflow; NLP engineering pipeline; transfer-learning pipeline.

IMPORTANT DEFINITIONS

Feature engineering, convolution, padding, stride, pooling, loss function, regularization, overfitting, transfer learning,

fine-tuning, backpropagation, optimizer, validation set, test set, tokenization, TF-IDF, embedding.

HIGH-PRIORITY LAST-MINUTE CONCEPTS

1. LSTM and GRU equations and gate functions.
2. CNN architecture, convolution, padding, stride, pooling, and output-size calculation.
3. Forward propagation, backpropagation, gradient descent, SGD, mini-batch GD, Adam.
4. RNN/LSTM/GRU comparison.
5. Transfer learning vs fine-tuning.
6. LeNet/AlexNet/VGG/Inception/ResNet differences.
7. NLP pipeline, preprocessing, feature representation, evaluation metrics, and optimization.
8. Loss functions and when to use them.
9. Regularization methods and overfitting.
10. Precision, recall, F1, confusion matrix, and accuracy.